Student Workbook

One chapter per week: ideas, pictures, exercises and new words

Tip: How to use this workbook

Bring this workbook to every session. Read the chapter with your partner, try every exercise, and fill in the reflection at the end. It is fine to write in it, draw in it and make mistakes in it!

Contents

Week 1: Save Points: Git Basics 2
Week 2: Objects: Blueprints and Things 6
Week 3: Families of Objects 9
Week 4: Searching and How Fast Is Fast? 12
Week 5: Sorting, Stacks and Queues 15
Week 6: Review and Checkpoint 18
Week 7: Grids, Recursion and Fractal Art 21
Week 8: Data Detectives 25
Week 9: How Do Machines Learn? 29
Week 10: Project Studio 1 34
Week 11: Project Studio 2 38
Week 12: Showcase and Graduation 42

Week 1 Level 3: Advanced

Save Points: Git Basics

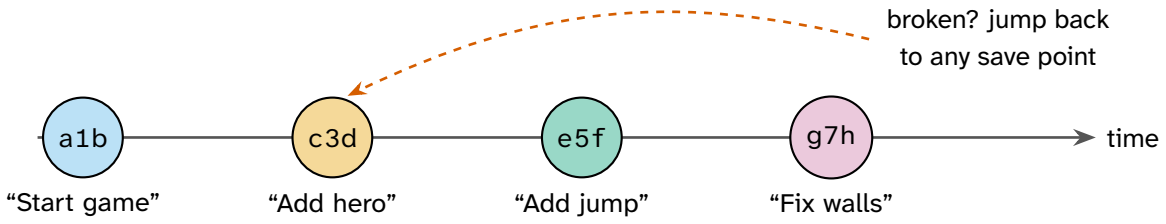
Git is a save-game system for code: every commit is a save point you can go back to.

What you will learn

- ☐ Use Terminal to move between folders.
- ☐ Make save points (commits) with git and give them helpful messages.
- ☐ Look back in time with `git log`, spot changes with `git diff`, and undo mistakes with `git restore`.
- ☐ Warm up your Level 2 skills: functions, lists and loops.

The big idea

In a video game, a save point lets you try something risky. If it goes wrong, you load the save and try again. **Git** gives your code save points. Each save point is called a **commit**. Every commit has a short id (like `a1b2c3d`) and a **message** that says what changed. Together the commits make a timeline: the story of your project.



Saving takes **two moves**, like taking a class photo:



Terminal survival kit

Terminal is a chat window with the computer: type a command, press Return, read the answer.

Type this	It means
pwd	Where am I? (print working directory)
`ls`	List the files in this folder.
`cd git-quest`	Go into the folder `git-quest`.
`cd ..`	Go up one folder.
Tab	Finish a long name for you. Type `cd git-q` and press Tab.
↑	Bring back the last command, so you do not retype it.

The six git commands

Command	What it does	Game words
`git init`	start a save system in this folder	new save file
`git status`	what changed since the last save?	pause menu
`git add hero_badge.py`	choose what goes in the next save	pack your backpack
`git commit -m "Add star"`	make a save point with a message	save the game
`git log --oneline`	list the save points, newest first	load-game menu
`git diff`	show exactly what changed	spot the difference
`git restore hero_badge.py`	throw away unsaved changes	reload last save

Tip: Stuck in a strange screen?

If you typed `git commit` without `-m`, git opens an editor called Vim. Press Esc, type `:q!` and press Return. Then try again with `-m "your message"`.

Good commit messages

A good message finishes the sentence "This commit will …". Start with a verb, keep it short (50 letters or fewer) and say *what* changed.

✗ Not helpful	✓ Helpful
`stuff`	`Add jump to hero`
`update`	`Make star twice as big`
`fixed it`	`Fix score going below zero`

Exercise 1.1: The Git Quest

Folder: `week01/ex-git-quest`. Follow the 7 levels in `README.md`: make a folder called `git-quest` in Documents, set your nickname, `git init`, make your first commit, change the badge color and look at `git diff`, break the file on purpose and rescue it with `git restore`, then make a commit of your own.

Done looks like: `git log --oneline` shows 3 or more commits with helpful messages.

Write your commit ids and messages here (newest first):

Exercise 1.2: Commit Coach

Folder: `week01/ex-commit-coach`. Finish the functions in `starter.py`:

- `is_good_message(message)` returns `False` for empty, too long, boring or one-word messages.
- `make_message("add", "star shape")` returns `"Add star shape"`.
- `find_secrets(lines)` returns the lines that contain words like `password` or `api_key`.

Check with `check_my_work.py`. Then Run it and test 3 messages of your own.

Exercise 1.3: Log Reader (if you finish early)

Folder: `week01/ex-log-reader`. Teach Python to read the text that `git log --oneline` prints. Each line is a commit id, a space, then the message:

```
line = "5b7a913 Fix hero walking through walls"
parts = line.split(" ", 1)   # split only at the FIRST space
```

```
print(parts)   # ['5b7a913', 'Fix hero walking through walls']
```

Finish `parse_log`, `count_commits`, `find_commits` and `oldest_first`. Extra: paste your own `git log --oneline` into SAMPLE_LOG.

Tip: Safety: history remembers everything

Git writes your `user.name` and `user.email` into every commit, forever. Use a nickname and an `@example.com` email. Never commit passwords, real names, addresses or photos of people. Deleting a file later does not delete it from the history.

Vocabulary box

git A program that keeps save points of a folder, so you can look back and undo.

repository (repo) A folder that git is tracking. Git keeps its notes in a hidden `.git` folder inside it.

commit A save point: a snapshot of your files plus a message and an id.

commit message A short label that says what changed, like “Add jump to hero”.

commit id The short code name of a commit, like `a1b2c3d`.

diff The differences between two versions: `-` lines were removed, + lines were added.

Terminal An app where you type commands for the computer.

Reflection

Why does git need two steps (`add`, then `commit`) instead of one?

Write the best commit message you made today, and why it is helpful:

Week 2 Level 3: Advanced

Objects: Blueprints and Things

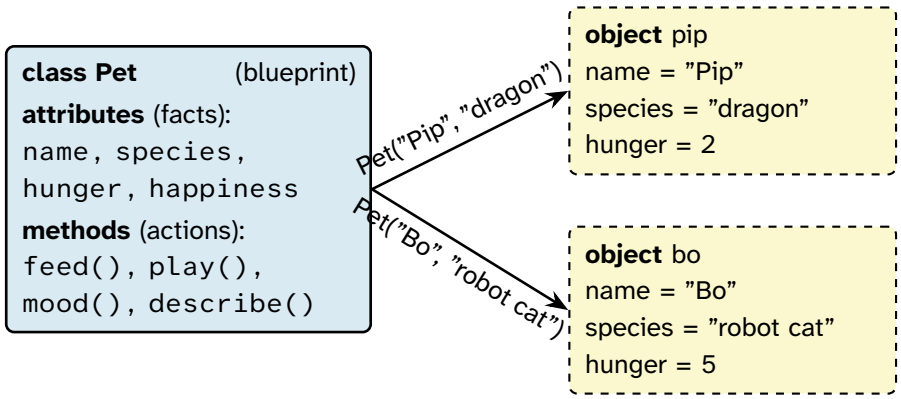
A class is a blueprint; objects are the things you build from it, and each one keeps its own data.

What you will learn

- ☐ Write a class with `__init__`, attributes and methods.
- ☐ Make many objects from one class and change one without changing the others.
- ☐ Use `self` correctly (and fix code that forgets it).
- ☐ Paint turtle art with a list of objects.

The big idea

A **class** is a blueprint, like a cookie cutter. An **object** is a thing made from the blueprint, like one cookie. Every object made from the same class has the same *kinds* of facts, but its own *values*.



Anatomy of a class

Every part has a job

```
class Pet:                          # 1. the blueprint (capital P)
    def __init__(self, name):       # 2. setup: runs when a pet is made
        self.name = name            # 3. attributes: facts it remembers
        self.hunger = 5
```

```
    def feed(self):                    # 4. a method: an action
        self.hunger = self.hunger - 3   # MY hunger goes down

pip = Pet("Pip")                       # 5. make an object
pip.feed()                             # 6. ask it to do something
print(pip.hunger)                      #    prints 2
```

Tip: What is self?

`self` means "this object" or "my". When you write `pip.feed()`, Python runs `feed` with `self` set to `pip`. So `self.hunger` is Pip's hunger. Every method needs `self` as its first parameter, and every attribute is written `self.something`.

From dictionary to class

In Level 2 you might have written `pip = {"name": "Pip", "hunger": 5}`. A dictionary holds the facts. A class holds the facts *and* the actions, all in one place, so you can make 100 pets that all know how to `feed()`.

Exercise 2.1: Pet Class

Folder: `week02/ex-pet-class`. `__init__` and `feed` are done. Finish:

- `play()`: happiness $+2$ (max 10) and hunger $+1$ (max 10).
- `mood()`: `"hungry"` if hunger ≥ 7, else `"happy"` if happiness ≥ 7, else `"bored"` if happiness ≤ 3, else `"okay"`.
- `describe()`: returns `"Pip the dragon feels okay."`

Predict first: after `pip.play()` twice, what are Pip's hunger and happiness? _____

What is Bo's hunger? _____

Exercise 2.2: Bubble Art

Folder: `week02/ex-bubble-art`. Each `Bubble` object remembers `x`, `y`, `size` and `color`. Finish `__init__`, `grow`, `move`, the big dot in `draw`, and `make_bubbles`, which returns a **list of objects**:

```
bubbles = make_bubbles(40)      # a list of 40 Bubble objects
for bubble in bubbles:
    bubble.draw(pen)            # each one draws itself
```

Done looks like: a black window full of bubbles, each with a white shine. Check with `check_my_`

`work.py`, then commit your picture's code in `git-quest`.

Tip: Kind design

Our pets never get sick, run away or say "you forgot me!". Good games are fun because you want to play, not because they make you feel guilty. When you design a game, ask: "Is this fair and kind to the player?"

Vocabulary box

class A blueprint that says what facts and actions a kind of thing has.

object One thing made from a class, with its own values. Also called an instance.

attribute A fact an object remembers, like `self.hunger`.

method A function that lives inside a class, like `feed(self)`.

__init__ The setup method ("dunder init"). It runs when a new object is made.

self Inside a method: "the object that is running this code", or "my".

Reflection

Pick something from a game you like. Write it as a class: its name, 3 attributes and 2 methods.

What does `self` mean, in your own words?

Week 3 Level 3: Advanced

Families of Objects

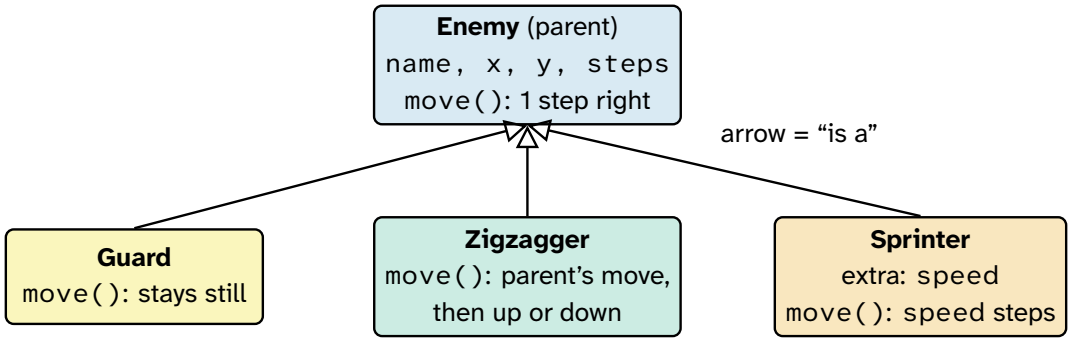
Child classes inherit from a parent and change only what is different; bigger objects are built from smaller ones.

What you will learn

- ☐ Make a child class that inherits from a parent class.
- ☐ Override a method, and use `super()` to reuse the parent's code.
- ☐ Call the same method on different objects and get different results (polymorphism).
- ☐ Build an object that holds a list of other objects (composition).

The big idea: inheritance

A **child class** gets everything from its **parent class** for free. Then it changes only what is different. Write the parent in brackets:



A child class

```
class Sprinter(Enemy):                  # a Sprinter IS AN Enemy
    def __init__(self, name, x=0, y=0, speed=3):
        super().__init__(name, x, y)    # parent sets name, x, y
        self.speed = speed              # then the extra attribute

    def move(self):                     # override the parent's move
        self.x = self.x + self.speed
        self.steps = self.steps + 1
```

Tip: super() = "do the parent's part"

`super().__init__(name, x, y)` runs the parent's setup. `super().move()` runs the parent's `move`, so the child can add something on top. If a child has its own `__init__` and forgets `super().__init__`, its parent's attributes are missing and you get an `AttributeError`.

Polymorphism: one call, many behaviors

```
enemies = [Enemy("Blob"), Guard("Rocky"), Sprinter("Zoom")]
for enemy in enemies:
    enemy.move()        # each one moves its OWN way
```

The loop does not need to know what kind each enemy is. *Polymorphism* means "many shapes": the same message, `move()`, gives different results.

Composition: "has a"

Not everything is a family. A Game is not a kind of Player, but a Game **has** Players. When an object keeps other objects in an attribute (often a list), that is **composition**.

Is a → inheritance	**Has a → composition**
A Sprinter is an Enemy.	A Game has Players.
A Star is a Shape.	A Party has Heroes.
`class Star(Shape):`	`self.players = []`

Exercise 3.1: Enemy Family

Folder: `week03/ex-enemy-family`. Finish `Zigzagger.move` (use `super().move()`), `Sprinter.__init__` (use `super().__init__`), `Sprinter.move` and `move_all`.
Predict: a Zigzagger starts at (0, 0). Where is it after 3 moves? _____

Exercise 3.2: Shape Family

Folder: `week03/ex-shape-family`. `Shape` and `Square` are done. Make `Triangle` (3 sides, `left(120)`) and `Star` (5 sides, `right(144)`) override `draw`. Fix `Polygon.__init__` with `super()`. Run it: one call to `draw_all` paints a whole grid of different shapes.

Exercise 3.3: Game Roster (if you finish early)

Folder: `week03/ex-game-roster`. A `Game` *has* a list of `Player` objects. Finish `add_player`, `give_points`, `leader` and `still_playing`.

Vocabulary box

inheritance A child class gets all the attributes and methods of its parent class.

parent class / child class The general class and the more special class built from it. Also called superclass and subclass.

override Write a method in the child with the same name, to replace the parent's version.

super() Inside a child class: "my parent". Use it to run the parent's version of a method.

polymorphism The same method call does different things for different classes.

composition An object that holds other objects, like a Game that has a list of Players.

Reflection

Draw a small family tree for a game you would like to make: one parent and two children. What does each child do differently?

Write one "is a" sentence and one "has a" sentence about your game:

Week 4 Level 3: Advanced

Searching and How Fast Is Fast?

Smart algorithms do less work: halving beats checking one by one.

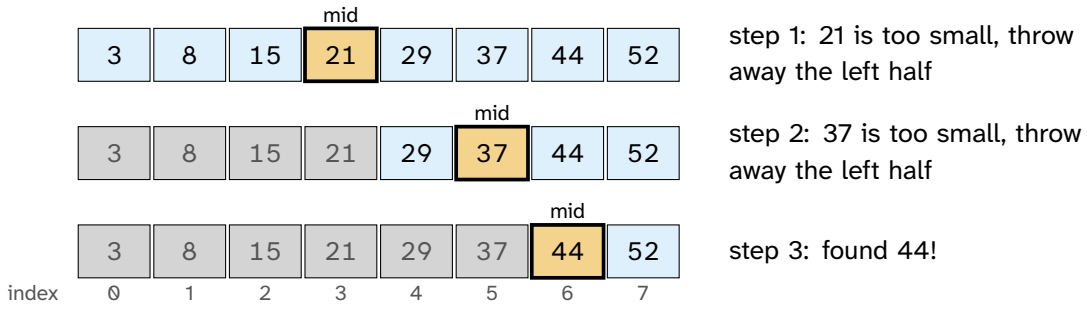
What you will learn

- ☐ Search a list in two ways: linear search and binary search.
- ☐ Count the steps an algorithm takes, and compare how they grow.
- ☐ Build a bot that always guesses your number in 7 tries or fewer.

Two ways to search

Linear search looks at the items one by one, from the start. It works on any list, but a long list means a long search.

Binary search only works on a **sorted** list. It looks at the middle item. If that is too small, the answer must be on the right, so it throws away the whole left half (and the other way round). Every step cuts the list in half.



Gray boxes have been thrown away; the thick orange box is the one we look at (`mid`). Searching for 44 took 3 steps. Linear search would take 7.

How fast is fast?

To compare algorithms, count the **steps** in the **worst case** (the unluckiest search).

Items in the list	Linear search	Binary search
10	10 steps	4 steps
100	100 steps	7 steps
1000	1000 steps	10 steps

When the list gets 10 times bigger, linear search does 10 times more work. Binary search only does about 3 more steps!

Binary search in Python

```
low = 0
high = len(items) - 1          # the LAST index, not the length
while low <= high:             # <= : one item left is still worth
    ↪ checking
    mid = (low + high) // 2    # // makes a whole number
    if items[mid] == target:
        return mid
    elif items[mid] < target:
        low = mid + 1          # throw away the left half (and mid)
    else:
        high = mid - 1         # throw away the right half (and mid)
return -1                      # nothing left: not in the list
```

Exercise 4.1: Search Race

Folder: `week04/ex-search-race`. `linear_search` is done. Write `binary_search`: it returns `(index, steps)`. Run it to see the race table.
Predict the binary steps for 10000 items: _____ What did the program say? _____

Exercise 4.2: Guess-the-Number Bot

Folder: `week04/ex-guess-bot`. First run the slow bot: it guesses 1, 2, 3... (press the red Stop button). Then:

- finish `check(guess, secret)`: return `"higher"`, `"lower"` or `"correct"`;
- make `next_guess(low, high)` return the middle.

Play 3 games. How many guesses did the bot need? _____ _____ _____

Tip: Filter bubbles

Search engines are amazingly fast, but they also choose which results to show you first, based on what you clicked before. If you only ever see what you already like, you are in a “filter bubble”. Try different search words, look at more than one source, and ask a grown-up when something seems odd.

Vocabulary box

algorithm A clear list of steps that solves a problem.

linear search Check items one by one from the start.

binary search In a sorted list, check the middle and throw away the half that cannot hold the answer.

worst case The unluckiest situation, when an algorithm needs the most steps.

off-by-one error A bug where a number is 1 too big or 1 too small, like using < instead of <=.

Reflection

Explain to a younger friend why binary search needs a sorted list:

Where in real life do you use halving to find something?

Week 5 Level 3: Advanced

Sorting, Stacks and Queues

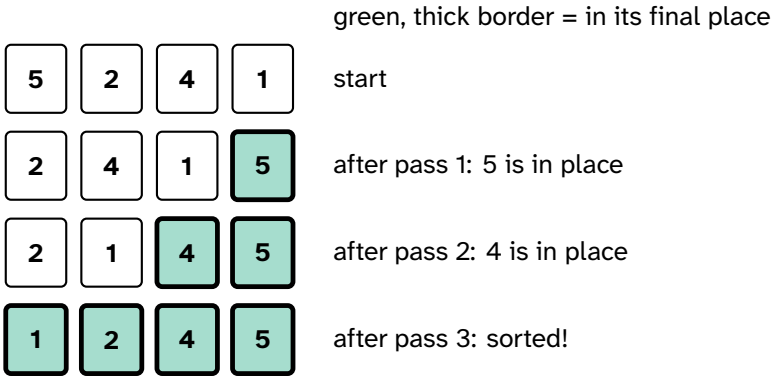
Sorting puts data in order; a stack remembers what to undo; a queue makes sure everyone gets a turn.

What you will learn

- ☐ Sort a list with bubble sort and selection sort, and count the swaps.
- ☐ Sort a leaderboard with `sorted(key=..., reverse=True)`.
- ☐ Use a stack to undo, and a queue to take turns.

Two ways to sort

Selection sort: find the smallest item and swap it to the front. Then find the smallest of the rest and swap it into second place. Repeat.
Bubble sort: walk along the list comparing neighbors. If the left one is bigger, swap them. After each pass the biggest item has “bubbled” to the end.



Python’s own sort

Python can sort for you. The **key** is a function that says *what to sort by*:

```
scores = [("Ada", 120), ("Leo", 95), ("Mei", 150)]

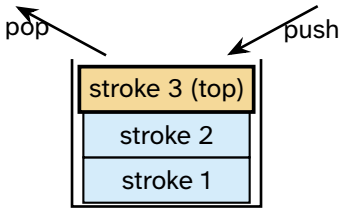
def points_of(entry):
```

```
    return entry[1]                    # sort by the number

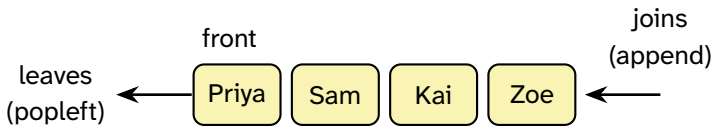
best_first = sorted(scores, key=points_of, reverse=True)
print(best_first)   # [('Mei', 150), ('Ada', 120), ('Leo', 95)]
```

`sorted` makes a **new** list and leaves the old one alone.

Stacks and queues



Stack: last in, first out



Queue: first in, first out

A **stack** is like a pile of plates: you can only take the top one. “Undo” uses a stack: the last thing you did is the first thing undone. A **queue** is like a lunch line: join at the back, leave from the front. In Python, `collections.deque` is a fast queue.

Exercise 5.1: Sort Race

Folder: `week05/ex-sort-race`. `selection_sort` is done. Finish the swap inside `bubble_sort`:

```
if result[i] > result[i + 1]:
    # swap both at once:
    result[i], result[i + 1] = result[i + 1], result[i]
    swaps = swaps + 1
```

Swaps for the 8 cards: selection _____ bubble _____

Exercise 5.2: Leaderboard

Folder: `week05/ex-leaderboard`. Finish `name_of` (sort A to Z, ignoring capitals), `top_scores` and the fair tie rule in `rank_lines`: equal points share a place, so the board reads 1, 2, 3, 3, 5.

Exercise 5.3: Undo Paint

Folder: `week05/ex-undo-paint`. Finish the `Stack` class: `push`, `pop`, `peek` and `size`. Then run it, click the window, draw with the arrow keys and press u to undo. Each stroke is one item on the stack.

Tip: Kind leaderboards

A good leaderboard uses nicknames, lets ties share a place, and celebrates personal bests and “most improved”, not only the winner.

Vocabulary box

sort Put items in order, for example smallest to largest.

swap Two items change places.

key function A function that tells `sorted` what to sort by.

stack A collection where the last item in is the first out (push and pop).

queue A collection where the first item in is the first out (join at the back, leave at the front).

deque Python’s fast queue, from `collections`. Say “deck”.

Reflection

Where do you see a stack and a queue in real life or in apps you use?

Why is “undo” a stack and not a queue?

Week 6 Level 3: Advanced

Review and Checkpoint

Connect the pieces, and use the debugger to watch your code think, one line at a time.

What you will learn

- ☐ Use the Thonny debugger to run a program one line at a time and watch the variables.
- ☐ Find and fix bugs in classes, sorts and searches.
- ☐ Show what you know in the mid-level checkpoint.

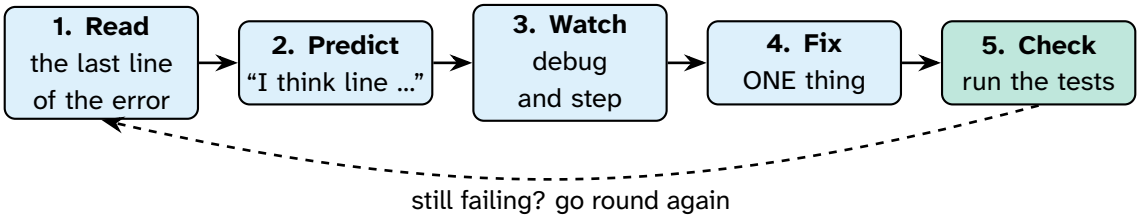
The Thonny debugger

A **debugger** is a slow-motion replay for your code. You can pause on every line and see the value of every variable.

Run	Debug	Step over	Step into	Resume	Stop
runs at full speed	starts slow motion (bug icon)	run this line, stop at the next	go inside a function call	run to the end or a breakpoint	end the program

- Turn on **View > Variables**. Click an object there to see its attributes.
- The highlighted line is the one that will run *next*.
- **Breakpoint:** click a line number to put a red dot on it. Press Debug, then Resume: the program runs and pauses at the dot.

A bug-hunting strategy



Tip: Reading an error

Start at the **bottom**. The last line says *what* went wrong (`TypeError`, `NameError`, `IndexError`…). The line above it says *where*: the file and line number.

Weeks 1–5 in one page

Week	Remember
1 Git	`git add` then `git commit -m "Say what changed"`; `log`, `diff`, `restore`.
2 Objects	`class`, `__init__`, attributes `self.x`, methods `def name(self):`.
3 Families	`class Child(Parent):`, `super().__init__(…)`, overriding; “has a” = composition.
4 Searching	linear: one by one; binary: sorted list, check the middle, halve.
5 Sorting	bubble and selection sort; `sorted(key=...)`; stack (undo) and queue (turns).

Exercise 6.1: Bug Hunt 1: Score Keeper

Folder: `week06/ex-bug-missing-self`. Three bugs, all the same kind. We debug this one together. Write each error and fix:

Exercise 6.2: Bug Hunt 2: Lost Scores

Folder: `week06/ex-bug-lost-items`. A sort that loses scores, and even empties the original list! Watch `scores`, `remaining` and `result` in the Variables view. Three bugs.

Exercise 6.3: Bug Hunt 3: Word Finder

Folder: `week06/ex-bug-binary-search`. Binary search with two off-by-one bugs. First trace the *buggy* code by hand for `find_word(["ant", "bee", "cat"], "ant")`:

low	high	low < high?	mid	words[mid] and so…

Why is “ant” never found? _____

Exercise 6.4: Bug Hunt 4: Shape Party

Folder: `week06/ex-bug-shape-party`. Three inheritance bugs that appear one after another: nothing drawn, then a crash, then diamonds that are only dots.

Vocabulary box

debugger A tool that runs your program in slow motion so you can watch every line and variable.

step over Run the highlighted line in one go, then pause.

step into Go inside the function or method that the line calls.

breakpoint A marked line where the debugger pauses.

traceback The error report Python prints. Read it from the bottom.

trace table A paper table where you write each variable’s value, step by step.

Reflection

My hardest bug today was … and what helped me find it was …

The topic I feel strongest in, and the one I want to practice:

Week 7 Level 3: Advanced

Grids, Recursion and Fractal Art

Functions that call themselves; worlds made of grids.

What you will learn

- What **recursion** is: a function that calls itself on a smaller problem.
- Why every recursive function needs a **base case**.
- How to store a game map or a picture in a **grid** (a list of lists).
- How to draw a **fractal tree** and build a **paint bucket** tool.

The big idea: functions that call themselves

Open a Russian doll and you find a smaller doll. Open that one: an even smaller doll. You keep doing *the same job* until you reach the tiny solid doll – then you stop.

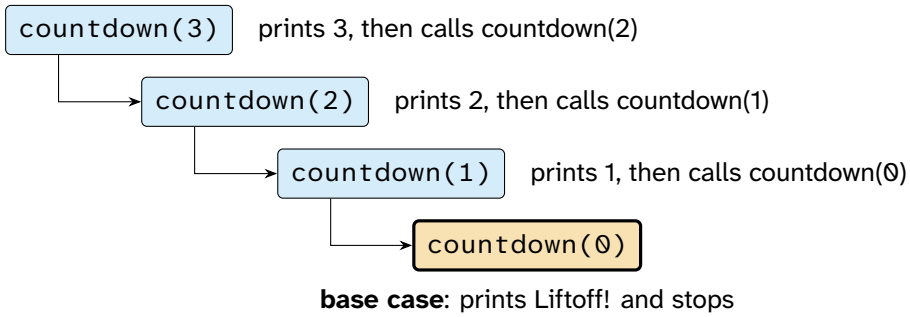
A **recursive** function works the same way. It has two parts:

- the **base case**: the tiny problem you can answer straight away – *stop here*;
- the **recursive case**: do one small step, then call yourself on a *smaller* problem.

countdown: a recursive function

```
def countdown(n):
    if n <= 0:              # base case: stop
        print("Liftoff!")
        return
    print(n)                # one small step
    countdown(n - 1)        # the same job on a smaller number
```

When you call `countdown(3)`, Python stacks up the calls like plates. Each call waits for the smaller one to finish:



Bug hunt: No base case = never-ending dolls

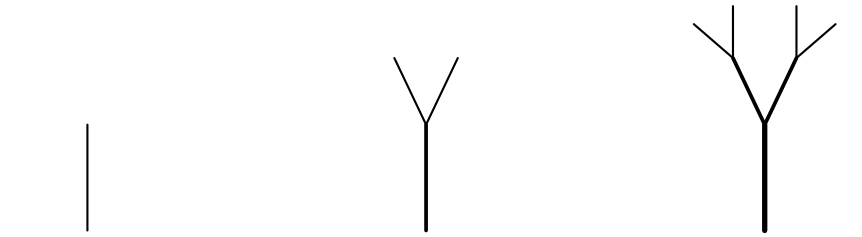
If you forget the base case, the function calls itself forever. Python stops it after about 1000 calls with `RecursionError: maximum recursion depth exceeded`. When you see that error, look for the base case first. Also check that each call is on a *smaller* problem (`n - 1`, not `n`).

Predict! What does `stars(3)` print? Write it before you run it.

```
def stars(n):
    if n == 0:
        return
    print("*" * n)
    stars(n - 1)
```

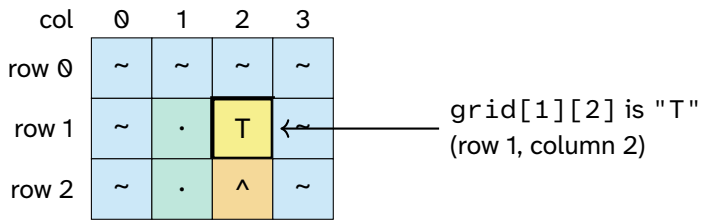
Fractals: pictures made of smaller copies

A **fractal** is a shape made of smaller copies of itself. A tree is a trunk with two smaller trees on top. Each smaller tree is a trunk with two even smaller trees on top… until the **depth** reaches 0.



Grids: worlds made of rows

A **grid** is a list of rows, and each row is a list of cells. You find a cell with `grid[row][col]`: first go *down* to the row, then *across* to the column. Both start at `0`.



You can make a grid from strings: `grid = [list(row) for row in ["~~~~", "~.T~", "~.^~"]]`. Change a cell with `grid[2][1] = "T"`. The **neighbors** of a cell are the cells up, down, left and right of it (watch out for the edges!).

A **paint bucket** (flood fill) uses recursion on a grid: paint this cell, then ask each neighbor to do the same – but stop at the edge, at walls, and at cells you have already painted. Those are the base cases.

Exercise 7.1: Recursion warm-up

Folder: `week07/ex-countdown-sum`. Read `countdown()`. Then finish `sum_to(n)` (so `sum_to(5)` is 15) and `reverse(word)` (so `reverse("turtle")` is `"eltrut"`) using recursion. Check with `check_my_work.py`.

Write the base case of `sum_to`:

Exercise 7.2: Fractal tree

Folder: `week07/ex-fractal-tree`. Inside `branch()`, after the trunk, add: turn left, draw a smaller tree, turn right twice as far, draw another smaller tree, turn back. Run it and fill in the table.

Depth	1	2	3	4	5
Number of lines	1	3	____	____	____

What pattern do you see?

Exercise 7.3: Paint bucket

Folder: `week07/ex-paint-bucket`. Finish `count_tiles(grid, symbol)` with two loops, then the flood fill: add the base case “stop if this cell is not the old color” and four recursive calls (up,

down, left, right). Run it, press keys **1–5** to choose a paint and click inside a shape. Every paint has a letter as well as a color.
Which cells does the paint stop at?

Tip: Watch the calls pile up

In Thonny, click the bug icon (Debug) and press F7 (Step into) again and again on `sum_to(3)`. Each new call opens a new box – that is the call stack.

Vocabulary box

recursion When a function calls itself to solve a smaller copy of the same problem.

base case The simplest case, answered without calling the function again. It stops the recursion.

recursive case The part that does a small step and calls the function on a smaller problem.

RecursionError Python's error when a function calls itself too many times – usually a missing base case.

fractal A shape made of smaller copies of itself, like a tree, a fern or a snowflake.

grid (2D list) A list of rows, where each row is a list. `grid[row][col]` is one cell.

flood fill Filling every touching cell of the same color, like a paint bucket tool.

Reflection

Where in nature have you seen something made of smaller copies of itself?

Explain the base case to someone who has never coded:

Week 8 Level 3: Advanced

Data Detectives

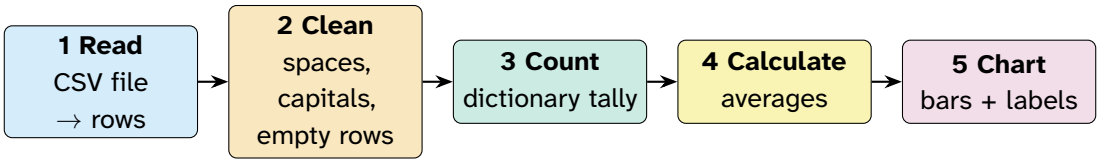
Data tells stories when you clean it, count it and chart it.

What you will learn

- How to read a **CSV file** with Python's `csv` module.
- How to **clean** messy data: extra spaces, MiXeD capitals, empty rows, missing numbers.
- How to **count** with a dictionary and work out an **average**.
- How to draw a **bar chart** that everyone can read – and how to keep survey answers private.

The big idea: the data detective's pipeline

A data detective always works in the same order. Skipping a step gives wrong answers!



1. Read: what is a CSV file?

CSV means *comma-separated values*: a table saved as plain text. The first line is the **header** (the column names). Every other line is one **row**. Here is the start of `favorite_games.csv` – made-up, anonymous survey answers (a response number, no names):

```
response,age_group,favorite_game,genre,hours_per_week,stars
1,10-11,Block Craft,building,5,5
2,12-13, block craft ,building,3,4
5,,,,,
7,12-13,Kart Dash,racing,,3
```

`csv.DictReader` turns each row into a **dictionary**: `row["favorite_game"]` is `"Block Craft"`.

```
import csv
with open("favorite_games.csv", newline="") as f:
```

```
    for row in csv.DictReader(f):
        print(row["favorite_game"], row["stars"])
```

2. Clean: wash the data first

Every value arrives as **text**, even numbers: `"5"`, not 5. And people type things in different ways. Spot the mess:

Messy value	Problem	Clean it with
`" block craft "`	extra spaces, small letters	`.strip().title()` → `"Block Craft"`
`"BLOCK CRAFT"`	capitals	`.title()` → `"Block Craft"`
`"5"`	a number stored as text	`int("5")` → 5
`""` (hours)	missing answer	use `None`, and skip it in averages
`5,,,,,`	empty row	skip the whole row

3 and 4. Count and calculate

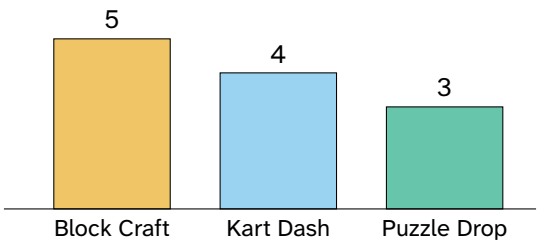
Counting with a dictionary is like a tally chart. The first time you see a game, `counts.get(game, 0)` gives `0`:

```
counts = {}
for row in rows:
    game = row["favorite_game"]
    counts[game] = counts.get(game, 0) + 1
# {"Block Craft": 5, "Kart Dash": 4, ...}
```

The **average** (mean) is the total divided by how many: `sum(numbers) / len(numbers)`. Leave out missing answers – counting a blank as `0` would make the average unfairly small.

5. Chart: readable by everyone

A good chart never relies on color alone. Write the **name** under each bar and the **number** on top, so someone who sees colors differently can still read it.



Exercise 8.1: Survey cleaner

Folder: `week08/ex-survey-cleaner`. Open `favorite_games.csv` in Thonny and circle the mess you find. Then finish `to_number()`, `is_empty()` and add `.title()` to the game names. **Done:** `Clean answers: 17`.

How many rows did the cleaner throw away, and why?

Exercise 8.2: Survey stats

Folder: `week08/ex-survey-stats`. Finish `count_by()`, `average()` and `most_popular()`. Then answer from your program's report:

Most popular game: _____ Average hours per week: _____

Adventure games got 5.0 stars on average from only 2 answers. Should we trust that? Why?

Exercise 8.3: Bar chart

Folder: `week08/ex-bar-chart`. Make `bar_height()` scale the bars so the biggest is `MAX_HEIGHT` pixels, and write a number on top and a name under every bar. **Done:** six labeled bars from Block Craft (5) to Space Chess (1).

Digital citizenship & safety: Consent and privacy in surveys

Before collecting data, ask: *Did people agree to answer? Can they skip a question? Do I really need this question?* Never collect real names, addresses, schools, birthdays or photos. Even without names, answers together can point to one person ("the only 13-year-old in the chess club"). Before sharing data, **anonymize** it: keep only the columns you need and give each row an ID like R1, R2.

Vocabulary box

CSV Comma-separated values: a table saved as plain text, one row per line.

header The first line of a CSV file, with the column names.

data cleaning Fixing messy data (spaces, capitals, missing or empty values) before using it.

average (mean) The total divided by how many values there are.

sample The group of people who answered. A small sample can give misleading results.

consent Agreeing, freely, to take part – with the choice to say no.

anonymize Remove or replace anything that could show who gave an answer.

Reflection

What story does the bar chart tell? What can it *not* tell us?

Would you answer a survey that asked for your full name? Why or why not?

Week 9 Level 3: Advanced

How Do Machines Learn?

AI finds patterns in examples – and copies their mistakes.

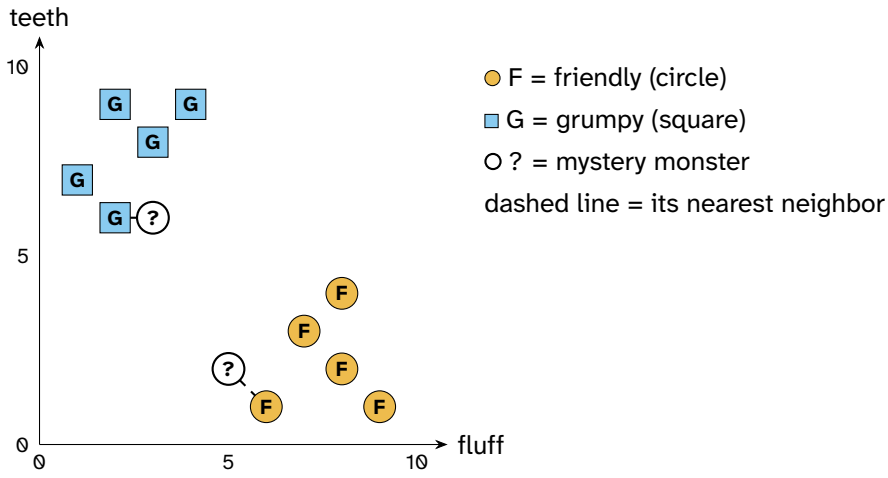
What you will learn

- The difference between a program that follows **rules** and one that **learns from examples**.
- How a **nearest-neighbor classifier** sorts things, and how to measure its **accuracy**.
- Why unbalanced or wrong examples make an AI unfair (**bias**).
- How a tiny text generator writes sentences – and why chatbots can be confidently wrong.

Rules or examples?

A program with **rules** does exactly what you typed: `if teeth > 5: print("grumpy")`. A **learning** program gets lots of **examples** with the right answers (**labels**) and finds the pattern itself. The things it measures are called **features**.

Our monsters have two features: **fluff** (0–10) and **teeth** (0–10). Put every example on a chart. To guess a **mystery** monster, find the closest example and copy its label. That is a **nearest-neighbor classifier**.



The mystery at (5, 2) is closest to a friendly monster, so the guess is *friendly*. Distance is measured with Pythagoras: `math.sqrt(dx * dx + dy * dy)`.

Is it any good? Train, then test

Split the examples into two piles. The program learns from the **training** pile. The **test** pile stays secret until the end; then we count how many test cards it labels correctly.

$$\text{accuracy} = \frac{\text{correct guesses}}{\text{number of test cards}}$$

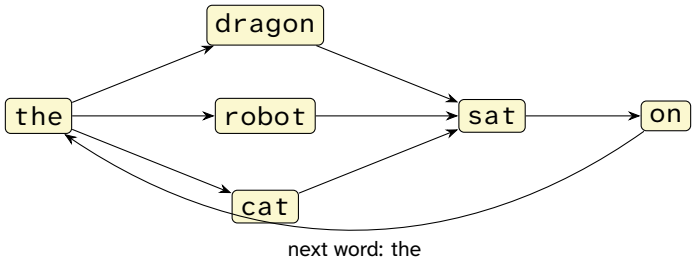
Testing on cards it learned from would be like seeing the exam answers first.

Bug hunt: Bad examples, bad predictions

If the training pile has 8 grumpy monsters and only 1 friendly one, the program calls most friendly monsters grumpy. The code is fine – the *data* is the bug. This is called **bias**. When real AI decides things about people (faces, voices, job applications), biased examples can treat whole groups unfairly. That is why people must check the data, not just the code.

A tiny text generator

A **Markov chain** learns just one thing from a text: *after each word, which words came next?*



To write, it starts with a word and keeps picking a *random* next word from the list. The sentences *sound* like the training text, but the machine does not understand them at all. Today's chatbots are enormously bigger and cleverer, but they also predict likely next words – which is why they can sound sure and still be wrong.

The five big ideas of AI

Big idea	What it means	Today
1 Perception	Computers sense the world with cameras, microphones, sensors.	a camera measuring fluff?
2 Representation & reasoning	The world is stored as data the computer can reason about.	each monster = (fluff, teeth)
3 Learning	Computers find patterns in examples.	nearest-neighbor classifier
4 Natural interaction	Talking and writing like people is very hard.	Markov story machine
5 Societal impact	AI can help or harm people.	bias, privacy, deepfakes

Exercise 9.1: Monster sorter

Folder: `week09/ex-monster-sorter`. Finish `distance(a, b)` and `nearest(training, mystery)`. Run it and look at the chart. What does it guess for (5, 5)? Do you agree?

Exercise 9.2: Fair or biased?

Folder: `week09/ex-fair-or-biased`. Finish `label_counts()` and `accuracy()`. Fair training accuracy: _____ Biased training accuracy: _____ Why is the biased one worse?

Exercise 9.3: Markov story machine

Folder: `week09/ex-markov-story`. Finish `build_chain(words)`: each word maps to the list of words that came right after it (keep repeats!). Copy your silliest generated sentence:

Exercise 9.4: My project pitch (Milestone 1)

For the next three weeks you will *Build Something Big*. Read the Project brief, then fill in your pitch. Finish it at home.

Project title: _____

Option (circle one): grid game generative art app data/AI app

In one sentence, it is...: _____

Two classes I will need: _____

Inheritance or composition: _____

Data structure (dict / stack / queue / grid): _____

Named algorithm: _____

File it will save: _____

The smallest fun version: _____

Digital citizenship & safety: Using AI wisely and safely

AI can be wrong, even when it sounds sure – check important facts with a trusted source or adult. **Never share personal information** with a chatbot: no real name, address, school, passwords, photos or secrets; what you type may be stored. **Deepfakes** are fake pictures, voices or videos made by AI: ask "Who made this, and why?" before you believe or share, and never make fakes of real people. **Bias**: AI is only as fair as its examples.

Vocabulary box

feature Something we measure about an example, like fluff or teeth.

label The right answer for an example, like "friendly".

training data The labeled examples a program learns from.

test data Examples kept secret during training, used to check the guesses.

accuracy Correct guesses divided by the number of test examples.

classifier A program that sorts things into groups (labels).

bias When unbalanced or unfair data makes a program treat some cases unfairly.

deepfake A fake photo, voice or video made with AI.

Reflection

Where have you met AI this week (videos, games, phones, homework)?

One thing you will never type into a chatbot, and why:

Week 10 Level 3: Advanced

Project Studio 1

Design first: classes, data and milestones – then build the core.

What you will do

- Design your project on paper with **CRC cards** and a **class diagram**.
- Plan your **milestones**: small steps you can finish and save.
- Start a **git repository** for your project and make your first commits.
- Check that every picture, sound or library you use is **allowed**.

Design first

Builders draw plans before they pour concrete. Programmers design before they type. Your design answers three questions: *Which classes do I need? What does each one know and do? Who works with whom?*

CRC cards

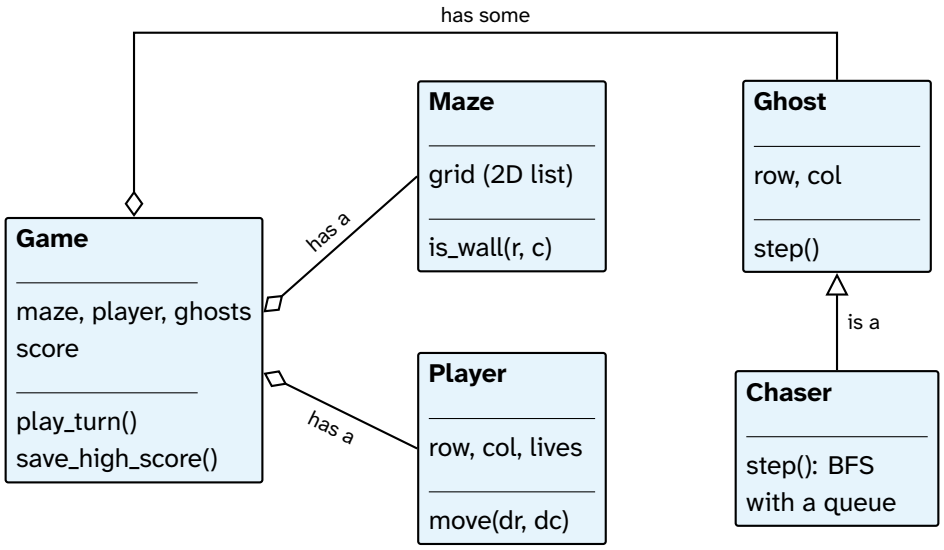
A **CRC card** is an index card for one class: the **C**lass name, its **R**esponsibilities (what it knows and does) and its **C**ollaborators (the classes it works with). Here is one from the reference project, *Maze Muncher*:

Class: Chaser (is a Ghost)	
Responsibilities – knows its row and column (from Ghost) – each turn, moves one step towards the player – finds the shortest path with breadth-first search and a queue	**Collaborators** Maze (where are the walls?) Player (where is it?)

Tip: if a card's responsibility list gets long, that class is doing too much – split it in two.

Class diagrams: “is a” and “has a”

A class diagram shows all the classes and how they connect. An arrow with a hollow triangle means **is a** (inheritance). A line with a diamond means **has a** (composition).



Read it aloud: *a Game has a Maze, a Player and some Ghosts; a Chaser is a Ghost.*

Milestones and save points

Big projects are built in small steps. After each step works, you **commit** – a save point you can always return to.



A good commit message starts with a verb and says what changed: `Add Maze class that loads a grid`, `Fix player walking through walls`. Your project needs **8 or more** meaningful commits by the showcase.

Exercise 10.1: My CRC cards

Write one card per class (at least two classes; one must use inheritance or composition). Copy them here.

Class	**Responsibilities** (knows / does)	**Collaborators**

Exercise 10.2: My class diagram

Draw a box for each class (name, what it knows, what it does). Join them with “is a” arrows and “has a” lines. Then write one sentence: … *is a* … or … *has a* ….

Exercise 10.3: My milestone plan

Milestone	**What will work**	**Done? (date)**
M1 Pitch	My idea and pitch card (week 9)	
M2 Core	My classes exist and the smallest version runs (week 10)	
M3 Features	+ 5 tests + a playtest (week 11)	
M4 Polish	+ README + accessibility fixes (week 11)	
Showcase	2-minute talk (week 12)	

Exercise 10.4: My repository and first commits

- ☐ I copied the starter folder to `Documents/coding-projects` and renamed it.
- ☐ I ran `git init` inside my project folder (I checked with pwd first).
- ☐ Each class has an `__init__` and empty methods, and the file runs without errors.
- ☐ I have at least 2 commits. My best message: _____

Digital citizenship & safety: May I use this? Licenses

Everything in your project must be yours or allowed. Python's standard library is free to use, and your own drawings and sounds are yours. For anything made by someone else, look at its **license**: **CC0** = free for anyone; **CC BY** = free if you give credit; "All rights reserved" or no license = **do not use**. Never download anything without a parent, and list every borrowed item in the "Credits" part of your README.

Vocabulary box

CRC card A card for one class: Class name, Responsibilities, Collaborators.

class diagram A drawing of classes as boxes, joined by "is a" and "has a" links.

composition "Has a": an object is built from other objects (a Game has a Maze).

milestone A small goal on the way to the finished project.

repository A project folder that git keeps a history of.

license The rules that say how you may use someone's work.

Reflection

Which class was hardest to describe? Why?

What will your next commit message be?

Week 11 Level 3: Advanced

Project Studio 2

Test it, playtest it, polish it, explain it.

What you will do

- Write at least **five pytest tests** for your own classes (Milestone 3).
- **Playtest** with a partner and sort the bugs you find (**triage**).
- Write a **README** and check your project is **accessible** (Milestone 4).

Tests: crash-test dummies for your code

A test is a tiny program that checks your program. Test the **logic** (moving, scoring, saving), not the drawing. Test **normal** cases and **edge** cases: the corner, the wall, the empty list, the last life.

Example: tests for a tiny Player

muncher_demo.py

```python
class Player:
    def __init__(self, row, col):
        self.row = row
        self.col = col
        self.score = 0

    def move(self, grid, d_row, d_col):
        new_row, new_col = self.row + d_row, self.col + d_col
        if grid[new_row][new_col] == "#":
            return False
        self.row, self.col = new_row, new_col
        if grid[new_row][new_col] == "*":
            self.score += 10
            grid[new_row][new_col] = "."
        return True
```

test_muncher_demo.py

```python
from muncher_demo import Player

```

```python
def make_grid():   # a tiny maze: wall, empty, star, empty, wall
    return [list("#####"), list("#.*.#"), list("#####")]

def test_player_moves_into_an_empty_cell():
    p = Player(1, 1)
    assert p.move(make_grid(), 0, 1) is True
    assert (p.row, p.col) == (1, 2)

def test_walls_block_the_player():          # edge case
    p = Player(1, 1)
    assert p.move(make_grid(), -1, 0) is False
    assert (p.row, p.col) == (1, 1)

def test_eating_a_star_scores_10_once():    # edge case
    grid = make_grid()
    p = Player(1, 1)
    p.move(grid, 0, 1)
    p.move(grid, 0, -1)
    p.move(grid, 0, 1)
    assert p.score == 10
```

How to run your tests: in Thonny choose **Tools** → **Open system shell...**, make sure you are in your project folder, and type `python -m pytest -q`. Each dot is a passing test; an F is a failing one, with its name and the line that failed.

Tip: Is my test any good?

Break your code on purpose. Does the test go red? If not, the test is not checking anything. Then fix the code and watch it go green.

Exercise 11.1: My five tests (Milestone 3)

Plan your tests, then write them in `test_<your project>.py`. At least one must be an edge case.

#	**Test name** (`test_...`)	**What it checks**	**Edge?**
1			
2			
3			
4			
5			

Playtest and triage

In a **playtest**, someone else uses your project while you watch **silently** and take notes. Every time they get stuck, you learn something. Afterwards, sort the notes into three piles – that is called **triage**:

1 Must fix	2 Should fix	3 Nice to have
crashes, cannot start, cannot win	confusing, unfair, hard to read	new ideas for version 2

For a crash: first write a test that shows the bug, then fix it. The test makes sure it never comes back.

Exercise 11.2: Playtest feedback cards

Cut these out (or copy them onto index cards). Fill in one card for each thing you notice while your partner plays.

What happened?

What did the player expect?

Pile: ☐ must ☐ should ☐ nice

What happened?

What did the player expect?

Pile: ☐ must ☐ should ☐ nice

What happened?

What did the player expect?

Pile: ☐ must ☐ should ☐ nice

What happened?

What did the player expect?

Pile: ☐ must ☐ should ☐ nice

README: explain it to a stranger

A README is the first thing people read. Future-you is a stranger too!

Exercise 11.3: My README (Milestone 4)

Create `README.md` in your project folder with these sections:

☐ **Title and one sentence**: what it is.

☐ **How to run it**: which file to open and press Run.

- ☐ **Controls**: keys or clicks.
- ☐ **How it works**: my classes (is-a / has-a), my data structure, my algorithm, the file it saves.
- ☐ **Tests**: how to run them, how many there are.
- ☐ **Credits**: my name as a nickname, and every borrowed thing with its license (or "everything made by me").

Exercise 11.4: Accessibility and usability check

- ☐ Colors come from the Okabe–Ito palette, and nothing depends on color alone (shapes, letters or text too).
- ☐ Text is big enough to read from a chair.
- ☐ The controls are written on the screen.
- ☐ Nothing flashes fast.
- ☐ A new player knows what to do within 10 seconds.
- ☐ One thing I fixed today: _____

Vocabulary box

edge case An unusual input at the limits: a corner, an empty list, zero, the last life.

playtest Watching someone use your project to find problems.

triage Sorting problems by how much they matter, to fix the important ones first.

README A file that explains what a project is, how to run it and how it works.

usability How easy something is to use for someone new.

accessibility Designing so that people with different abilities can use it too.

Reflection

What did your playtester do that surprised you?

Which test are you proudest of, and why?

Week 12 Level 3: Advanced

Showcase and Graduation

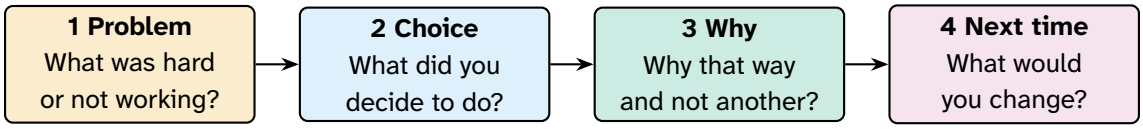
Share what you built, explain your choices, and plan what comes next.

What you will do

- Give a **2-minute design talk** about one decision in your project.
- Visit everyone's project in a **gallery walk** and give helpful feedback.
- Show what you have learned in the **exit assessment**.
- Look back at your journey and plan your **next steps**, safely.

The big idea: explain the *why*

Anyone can say *what* their program does. A real developer can explain *why* they built it that way. In two minutes, pick **one** decision and walk through four steps:



Example: "**Problem:** my first ghost moved randomly and the game was too easy. **Choice:** I made a Chaser class that *is a* Ghost, and its move uses breadth-first search with a queue. **Why:** inheritance meant I only wrote the new part, and BFS always finds the shortest path. **Next time:** the Chaser is too clever – it should only chase every other turn."

Good decisions to talk about: why you chose inheritance or composition, why a dictionary or a grid, which algorithm and why, what a test caught, what a playtester taught you.

Exercise 12.1: My showcase talk card

My project in one sentence: _____

1 Problem: _____

2 Choice: _____

3 Why: _____

4 Next time: _____

What I will show: ☐ playing it ☐ a class ☐ a test ☐ `git log`

Gallery walk: star and wish

Feedback helps when it is **kind**, **specific** and **useful**. "Cool!" is kind but not useful. "The score text was big and easy to read" is all three.

Exercise 12.2: Feedback cards

Write one card for each project you visit. Leave it face down next to the computer.

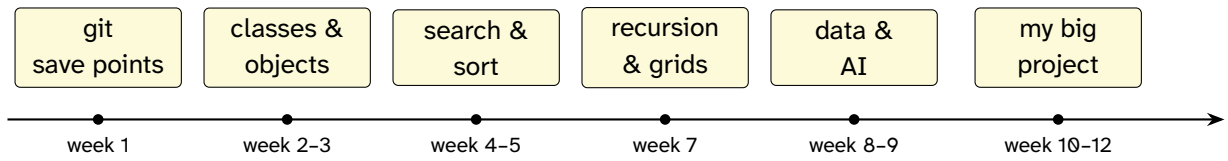
Project: _____
★ **Star** (I liked...):
Wish (What if...?):

Project: _____
★ **Star** (I liked...):
Wish (What if...?):

Project: _____
★ **Star** (I liked...):
Wish (What if...?):

Project: _____
★ **Star** (I liked...):
Wish (What if...?):

Look how far you have come



Exercise 12.3: My Level 3 journey

Run `git log --oneline` in your project. How many commits? _____ The skill I am proudest of:

Something that was hard at first and is easier now:

What next? Keep coding, safely

You can keep learning on your own. Some ideas: give your project a version 2; learn another Python library with a parent's help; try a new kind of program (a text adventure, a simulation, a puzzle solver); teach someone younger what you know.

Digital citizenship & safety: Next steps online

If you want to share your work or join a coding community, **choose it together with a parent**. Look for clear rules, moderators and a code of conduct. Use a **nickname** – never your real name, school, age, face or location, in the project or the account. Never move to private chats with strangers, never send photos, and tell an adult straight away if anything feels wrong. Your projects are yours: share only what you want to share.

Vocabulary box

design decision A choice about how to build something, with a reason.

feedback Comments that help someone improve: kind, specific, useful.

showcase An event where makers show and explain their work.

code of conduct The rules a community agrees to, so everyone stays safe and respected.

version 2 The next, improved release of a project.

Reflection

Which feedback card made you proudest?

What will you build next?